v0.57: Real-API PostgreSQL Role Isolation

11 November 2026 | Development checkpoint

PRODUCTION RELEASE: NO-GO

Previous CI integration smoke tests launched real API processes with the same PostgreSQL login used to create fixtures and apply migrations. That account was configured as a superuser. Since PostgreSQL superusers bypass row-level security even for FORCE RLS tables, those API tests could not establish that tenant isolation worked under a realistic runtime role.

Implemented security correction

• Separate **disposable CI operator** and **HTTP API runtime** database connection URLs; reject login reuse, remote hosts and mismatched databases before starting the API.

• Provision a distinct **NOSUPERUSER, NOBYPASSRLS, NOCREATEDB, NOCREATEROLE** CI login with a random password; do not log or store its credential in project source.

• Enable the existing runtime privilege and seven-table FORCE RLS startup checks during the supervised CI application run, as well as in production.

• Introduce a live read-only contract that checks current_user and its privileges, verifies RLS flags and table nonownership, and checks unscoped reads across the seven protected tables.

• Preserve the operator login for fixture/migration tasks only; pass the restricted URL exclusively to spawned API processes in two-server and HTTP integration tests.

Recorded offline evidence

Validation	Result
New v0.57 security tests	20/20 passed
Version suites v0.03-v0.57	55/55 passed
SQL migration files	25/25 unchanged
YAML/CI, architecture, syntax, source gate	Passed
Source snapshot secret scan	2,467 files, 0 high-confidence findings

The offline checks do not prove production safety

Installation and compilation

No authentic npm lockfile is present. npm registry DNS lookup failed with EAI_AGAIN, and npm ci refused to proceed. npm run build failed with missing Node and NestJS/pg type/module dependencies. More compile errors may appear once dependencies are installed.

Live security validation

No PostgreSQL server or installed pg module was available for the new runtime-role contract. Existing RLS migration, real two-API, HTTP integration, ClamAV malware-detection, authenticated browser and deployment checks were not executed here. No full Git-history credential scan or production-network/TLS/SSH review was completed.

Important scope limitation

The CI runtime role receives broad ordinary DML permissions over public tables so the existing integration suites can operate. It is **not** a production least-privilege grant specification, nor does seven-table RLS coverage imply complete tenant isolation. Remaining sensitive tables and service-layer authorization must still be verified. The runtime tenant setting is not a substitute for parameterized SQL and strict authorization checks.

Release actions that remain mandatory

• Generate and inspect a genuine package-lock.json using a connected, trusted Kali environment; run npm ci, production npm audit and a clean API/web build.

• Run the guarded GitHub Actions workflow against a fresh, disposable PostGIS database; prove the API uses the non-bypass role and complete the seven-table RLS and multi-server contracts.

• Pass live ClamAV and file-upload adversarial checks, login/MFA/session and cross-organization browser tests, and complete a full Git-history secret scan; rotate any exposed credentials.

• Validate release infrastructure, HTTPS, SSH restrictions, backups/restore, least-privilege grants, admin controls and deployment rollback; retain audit evidence and resolve critical findings.

Verdict: **NO-GO.** This artifact is an offline-tested source checkpoint and must not be used as proof of a secure commercial deployment.